In [24]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

1. The Setup & Loading Data

In [25]:
```
# df = pd.read_csv("C:\pp\ml--2026\project\WA_Fn-UseC_-Telco-Customer-Churn.csv")
df = pd.read_csv("C:\pp\ml--2026\project\WA_Fn-UseC_-Telco-Customer-Churn.csv")

# df = kagglehub.dataset_download("blastchar/telco-customer-churn")

df.head()
```

Out[25]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

5 rows × 21 columns

In [26]:
```
df.info()
```

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object 
 20  Churn             7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

In [27]:
```
df.shape
```

Out[27]:
```
(7043, 21)
```

In [28]:
```
df.dtypes
```

Out[28]:
```
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges       float64
Churn               object
dtype: object
```

In [29]:
```
df.isnull().sum()
```

Out[29]:
```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges       11
Churn               0
dtype: int64
```

In [30]:
```
df.describe()
```

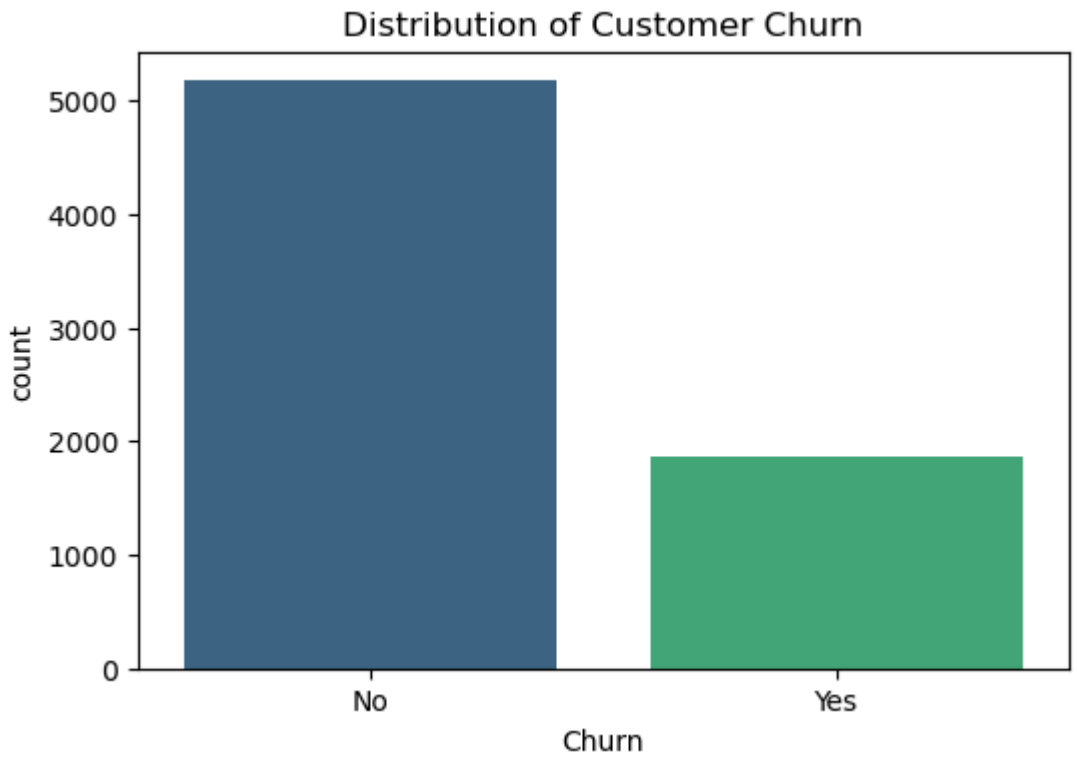
Out[30]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7032.000000
mean	0.162147	32.371149	64.761692	2283.300441
std	0.368612	24.559481	30.090047	2266.771362
min	0.000000	0.000000	18.250000	18.800000
25%	0.000000	9.000000	35.500000	401.450000
50%	0.000000	29.000000	70.350000	1397.475000
75%	0.000000	55.000000	89.850000	3794.737500
max	1.000000	72.000000	118.750000	8684.800000

In [31]:

```
plt.figure(figsize=(6, 4))
sns.countplot(x='Churn', data=df, palette='viridis')
plt.title('Distribution of Customer Churn')
plt.show()

# Check the percentage
print(df['Churn'].value_counts(normalize=True) * 100)
```




```
Churn
No     73.463013
Yes    26.536987
Name: proportion, dtype: float64
```

This dataset contains customer retention data from a telecommunications provider, comprising 7,043 unique customer records across 21 variables. It includes demographic details (gender, partner status), account information (tenure, contract type, payment method), and subscribed services (internet, security, streaming). Our goal is to predict customer churn (whether a customer will leave the company) using this historical data to identify risk factors and improve retention strategies.

2. Handling Missing Values & Duplicates

In [32]:
```
# Check for missing values
df.isnull().sum()
```

```
# Handle the few missing values created
df.dropna(inplace=True)

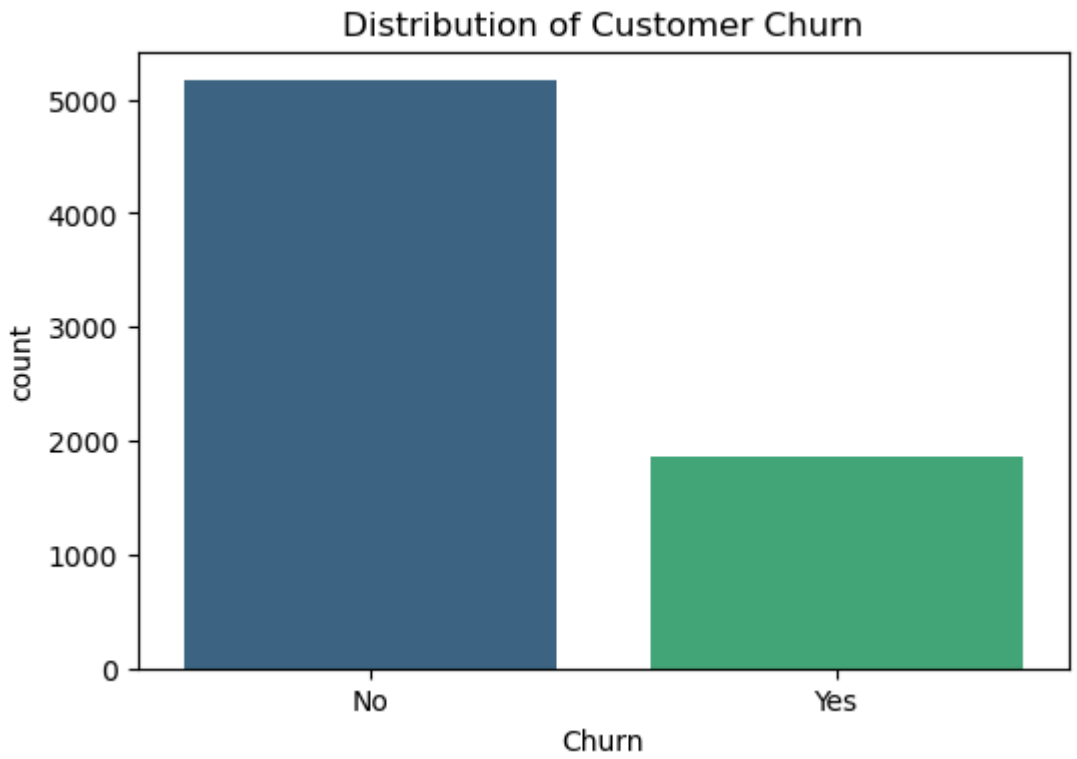
# Drop CustomerID (it's useless for patterns)
df.drop('customerID', axis=1, inplace=True)
```

The dataset contains no null values, ensuring complete data integrity across all features.

3. Analyzing the Target: Churn Rate

In [33]:
```
plt.figure(figsize=(6, 4))
sns.countplot(x='Churn', data=df, palette='viridis')
plt.title('Distribution of Customer Churn')
plt.show()

# Check the percentage
print(df['Churn'].value_counts(normalize=True) * 100)
```



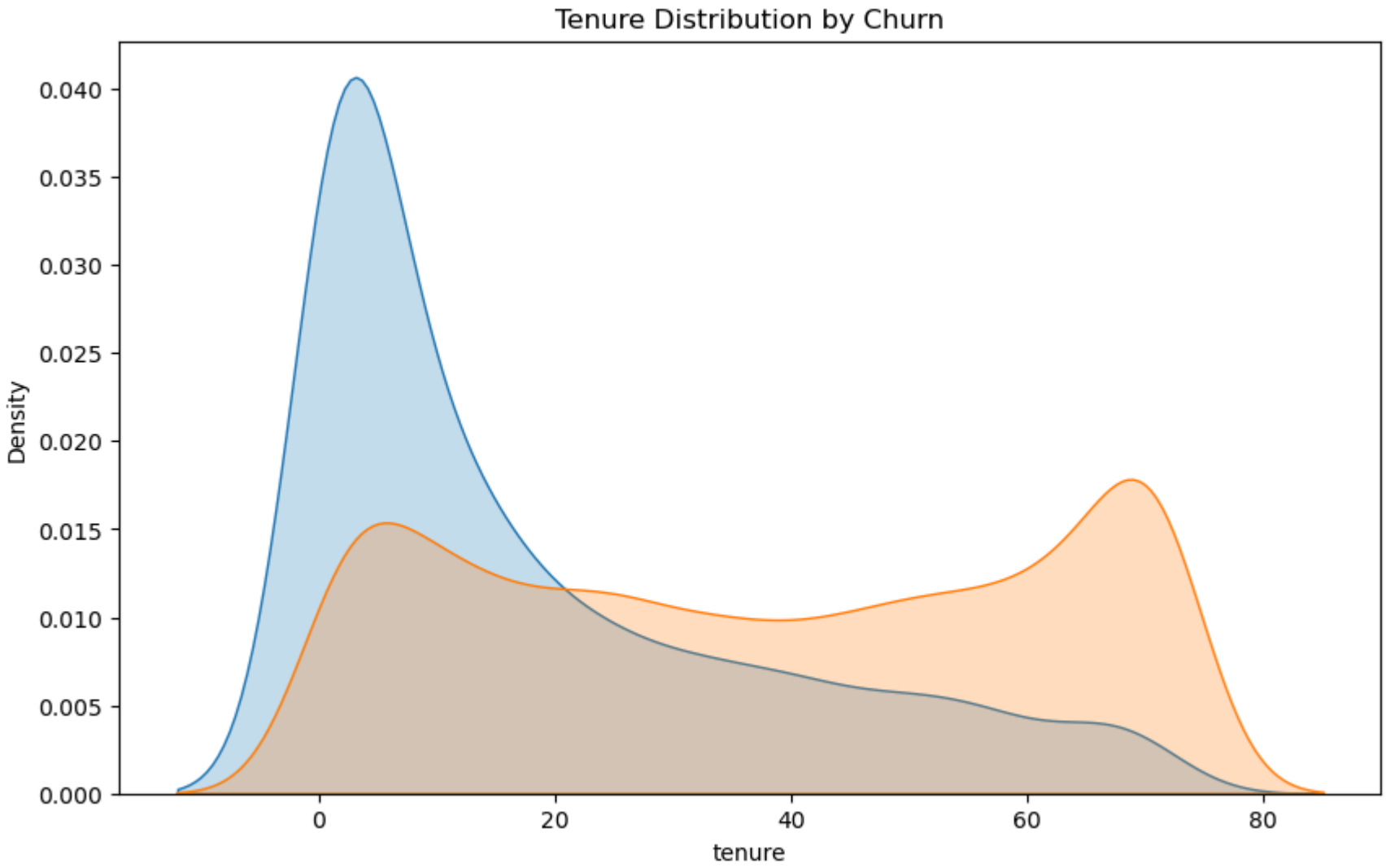

```
Churn
No     73.421502
Yes    26.578498
Name: proportion, dtype: float64
```

This shows that about one in four customers leave the company. Because the "Yes" group is smaller, we need to be very precise in identifying these specific individuals rather than just focusing on the majority who stay.

4. Numerical Features vs. Churn

In [34]:
```
# How does tenure (loyalty) affect churn?
plt.figure(figsize=(10, 6))
sns.kdeplot(df[df['Churn'] == 'Yes']['tenure'], label='Churn: Yes', shade=True)
sns.kdeplot(df[df['Churn'] == 'No']['tenure'], label='Churn: No', shade=True)
```

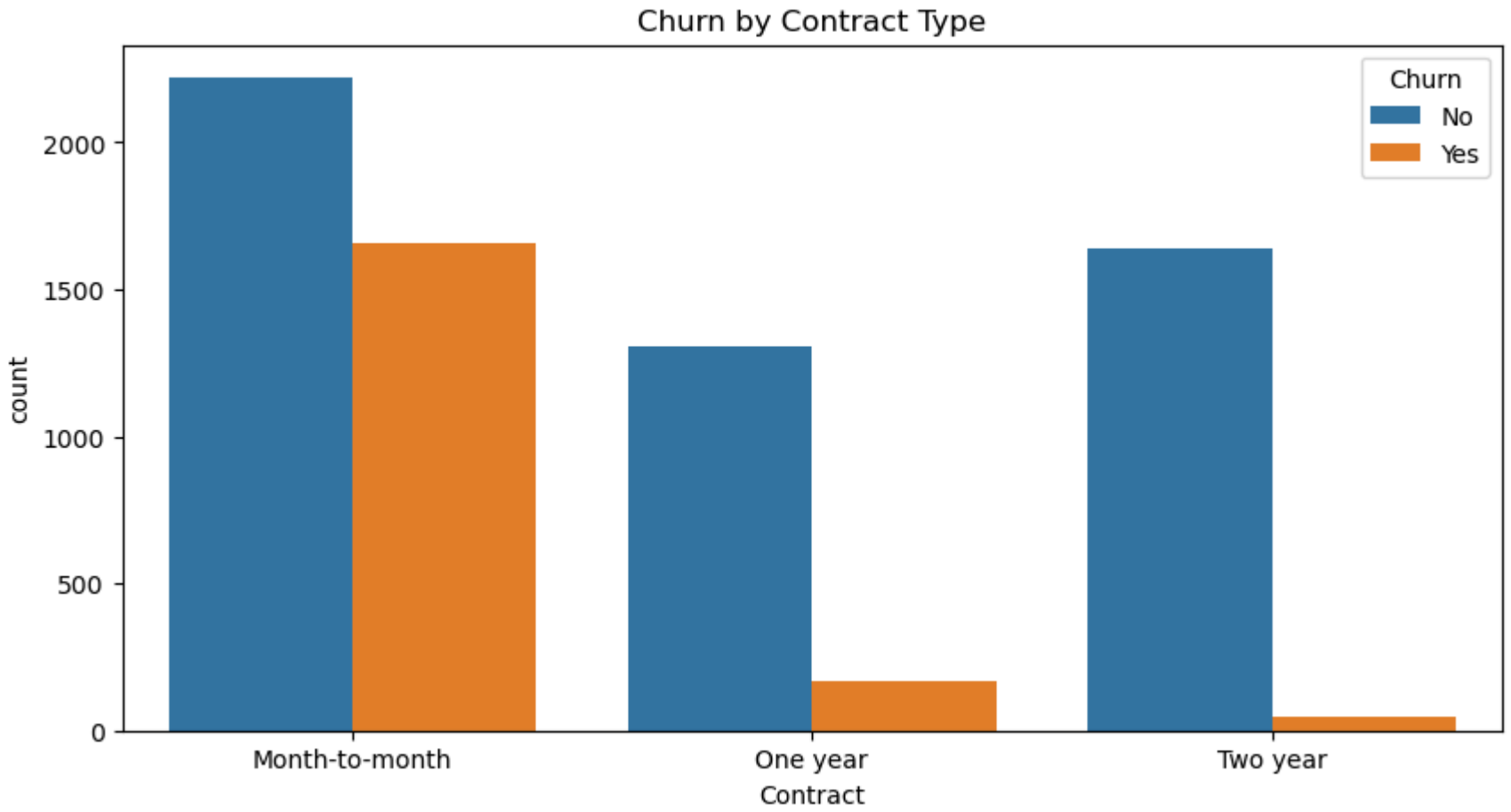
```
plt.title('Tenure Distribution by Churn')
plt.show()
```



There is a large spike in people leaving during their very first few months of service. This makes the length of time a customer has been with us a major clue for spotting which accounts are at the highest risk of exiting.

5. Categorical Features (The "Why")

In [35]:
```
# Compare Churn across Contract types
plt.figure(figsize=(10, 5))
sns.countplot(x='Contract', hue='Churn', data=df)
plt.title('Churn by Contract Type')
plt.show()
```

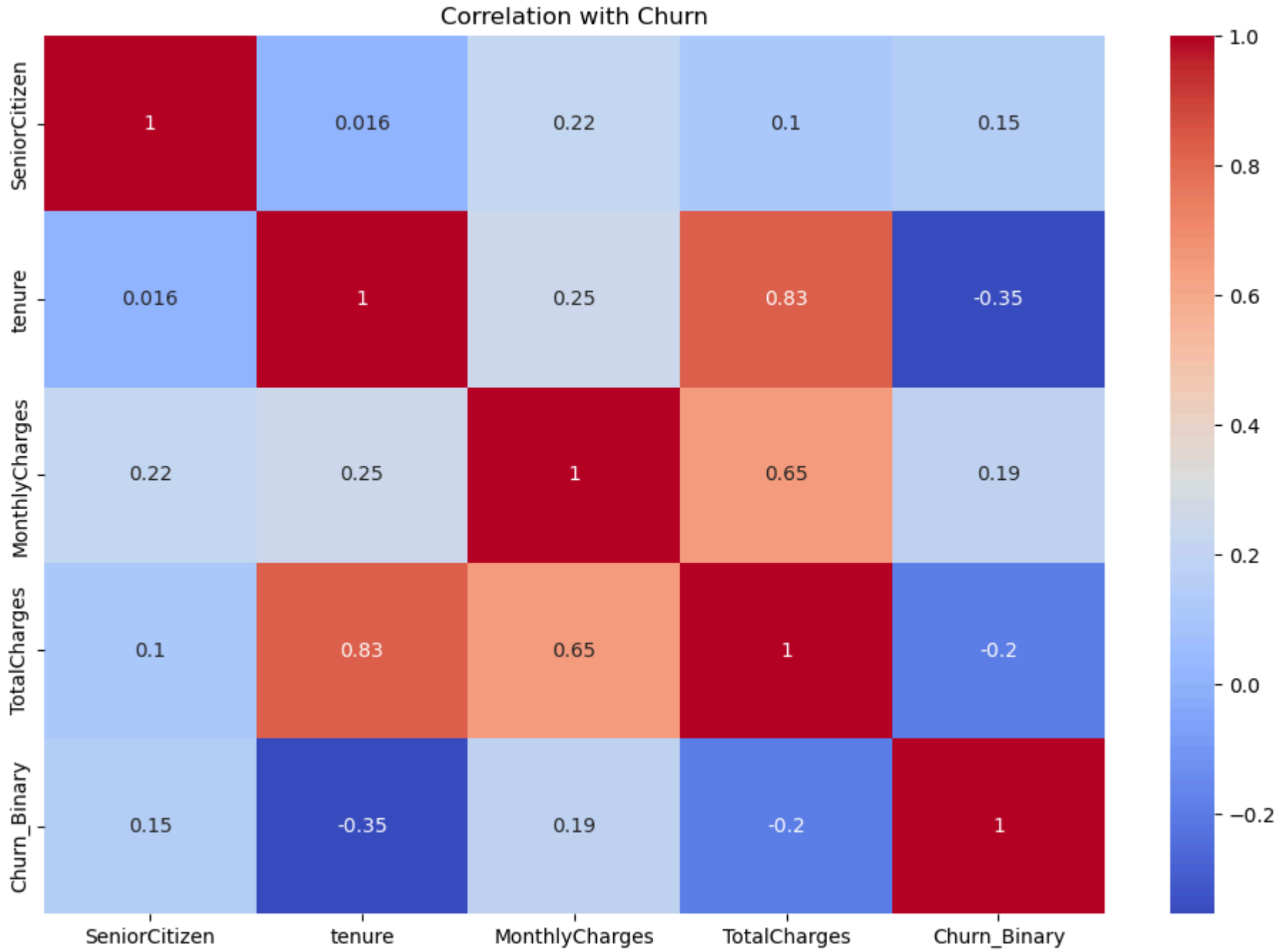


Customers on month-to-month plans leave at a much higher rate than those on one or two-year deals. This category acts as a strong signal to distinguish between loyal long-term customers and those who are likely to leave soon.

6. Feature Correlation

In [36]:
```
# Convert Churn to binary for correlation
df['Churn_Binary'] = df['Churn'].map({'Yes': 1, 'No': 0})

plt.figure(figsize=(12, 8))
# Correlate only numeric columns
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, cmap='coolwarm')
plt.title('Correlation with Churn')
plt.show()
```

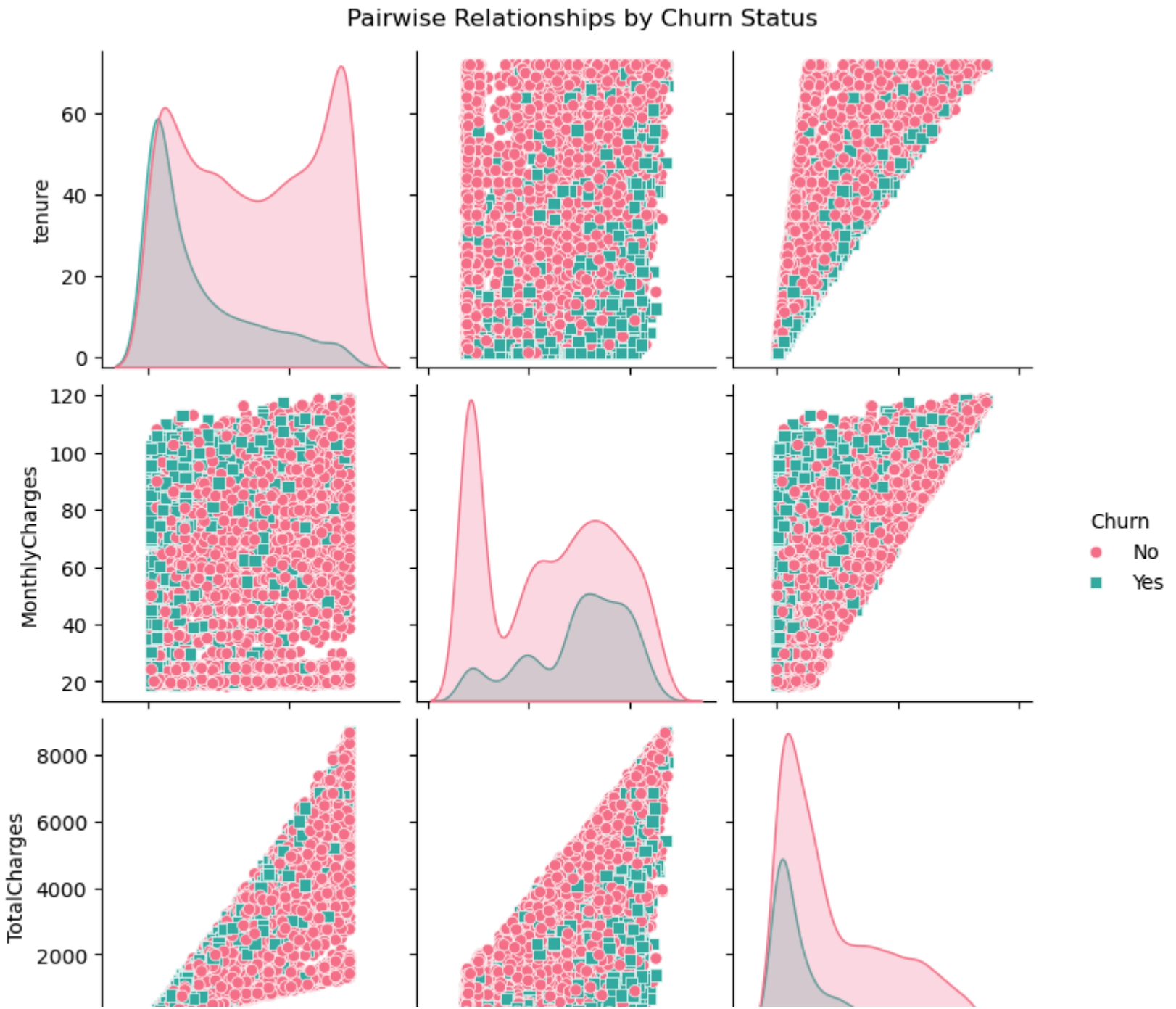

Correlation with Churn
SeniorCitizen
tenure
MonthlyCharges
TotalCharges
Churn_Binary
1
0.016
0.22
0.1
0.15
0.016
1
0.25
0.83
-0.35
0.22
0.25
1
0.65
0.19
0.1
0.83
0.65
1
-0.2
0.15
-0.35
0.19
-0.2
1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

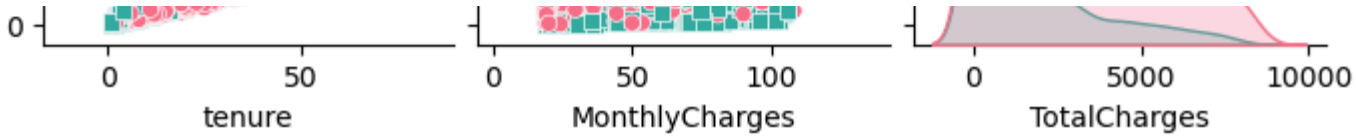
We see that as monthly bills go up, the likelihood of a customer leaving also increases. Tracking these price points helps us understand which specific costs are causing the most people to walk away.

7. Advanced Visualization: The "Pairplot"

In [37]:
```
# Select key numerical features and the target
numeric_cols = ['tenure', 'MonthlyCharges', 'TotalCharges', 'Churn']

# Pairplot to see clusters
sns.pairplot(df[numeric_cols], hue='Churn', palette='husl', markers=["o", "s"])
plt.suptitle('Pairwise Relationships by Churn Status', y=1.02)
plt.show()
```


Pairwise Relationships by Churn Status
tenure
MonthlyCharges
TotalCharges
0
20
40
60
20
40
60
80
100
120
2000
4000
6000
8000
Churn
No
Yes

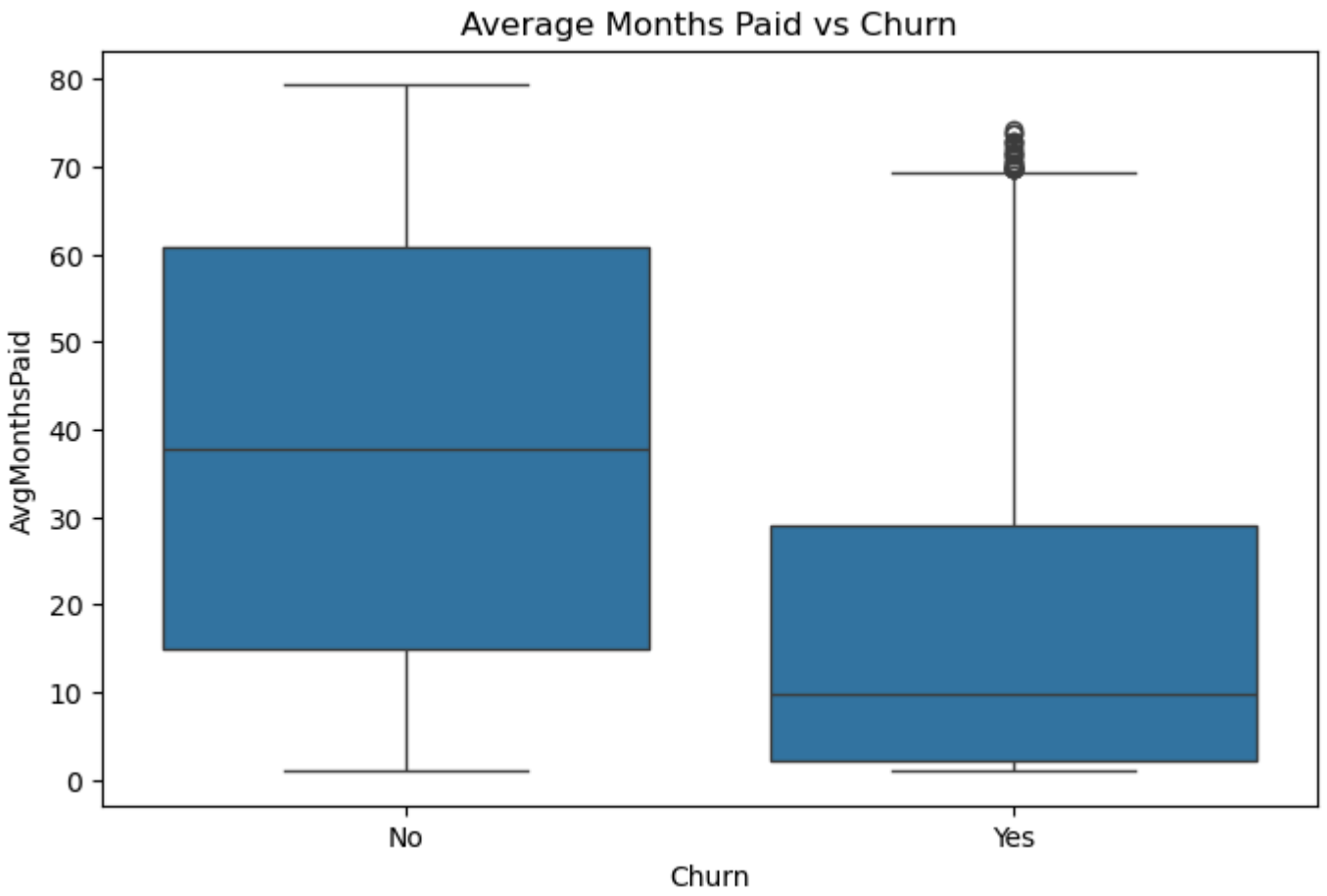


The pairplot shows clear separation between churned and non-churned customers in the relationship between MonthlyCharges and Tenure. Specifically, a "danger zone" is visible in the top-left of the scatter plots where tenure is low but monthly costs are high, identifying a high-risk customer segment.

8. Feature Engineering: "The Ratio"

In [38]:
```
# Creating a feature that represents 'Average Months Paid'
df['AvgMonthsPaid'] = df['TotalCharges'] / df['MonthlyCharges']

# Compare this new feature against Churn
plt.figure(figsize=(8, 5))
sns.boxplot(x='Churn', y='AvgMonthsPaid', data=df)
plt.title('Average Months Paid vs Churn')
plt.show()
```



This shows a significant gap between those who stay and those who leave. It provides a fresh way to separate the two groups based on how much they have actually invested relative to their monthly costs.

9. Encoding Categorical Variables

In [39]:
```
# 1. Target
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# 2. Clean TotalCharges
```

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors='coerce')
df = df.dropna()

# 3. Coding
df_final = pd.get_dummies(df, drop_first=True)

# 4. Checking
print(df_final.shape)
print(df_final.columns)
```

```
(7032, 33)
Index(['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges', 'Churn',
       'Churn_Binary', 'AvgMonthsPaid', 'gender_Male', 'Partner_Yes',
       'Dependents_Yes', 'PhoneService_Yes', 'MultipleLines_No phone service',
       'MultipleLines_Yes', 'InternetService_Fiber optic',
       'InternetService_No', 'OnlineSecurity_No internet service',
       'OnlineSecurity_Yes', 'OnlineBackup_No internet service',
       'OnlineBackup_Yes', 'DeviceProtection_No internet service',
       'DeviceProtection_Yes', 'TechSupport_No internet service',
       'TechSupport_Yes', 'StreamingTV_No internet service', 'StreamingTV_Yes',
       'StreamingMovies_No internet service', 'StreamingMovies_Yes',
       'Contract_One year', 'Contract_Two year', 'PaperlessBilling_Yes',
       'PaymentMethod_Credit card (automatic)',
       'PaymentMethod_Electronic check', 'PaymentMethod_Mailed check'],
      dtype='object')
```

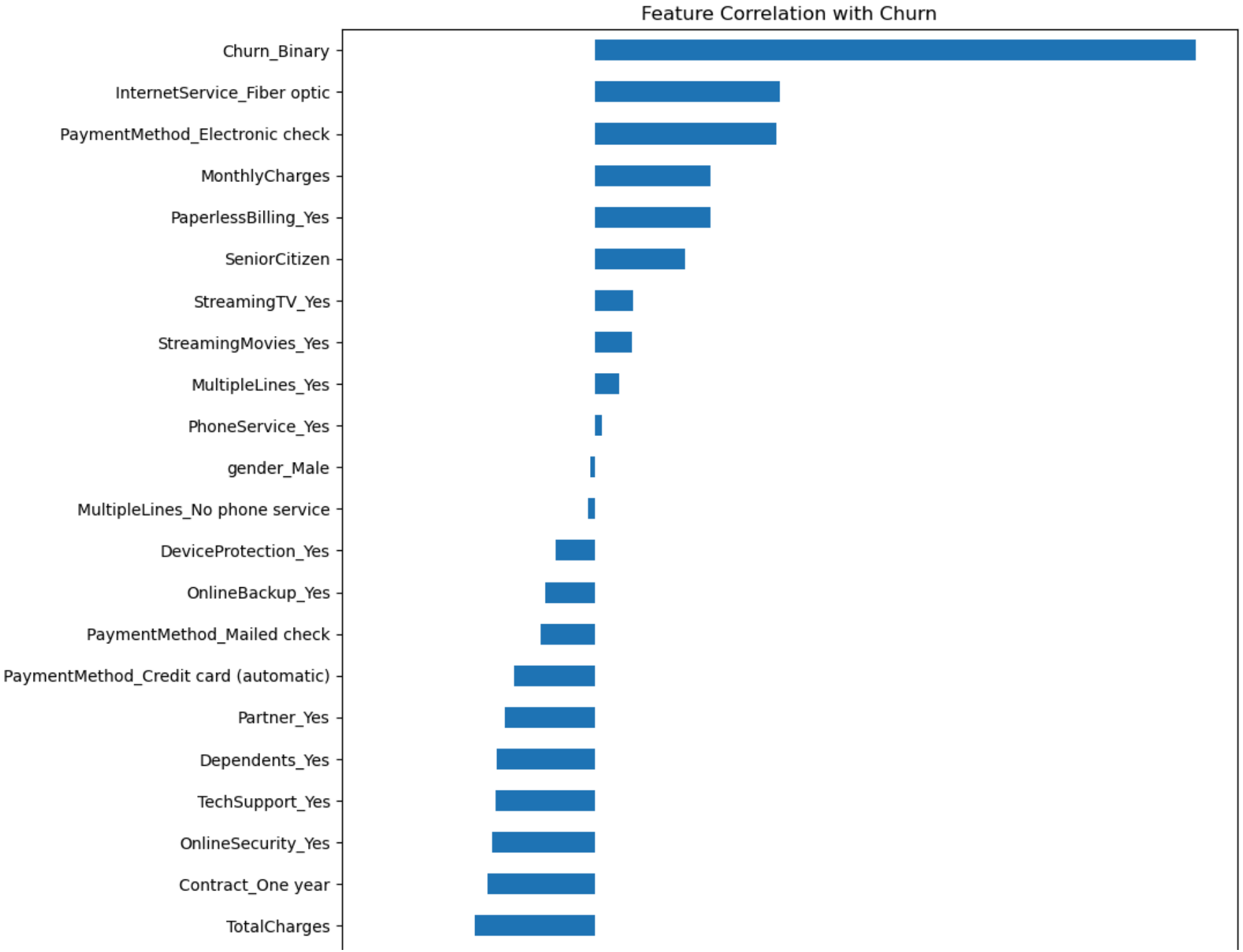
10. Final Correlation Check

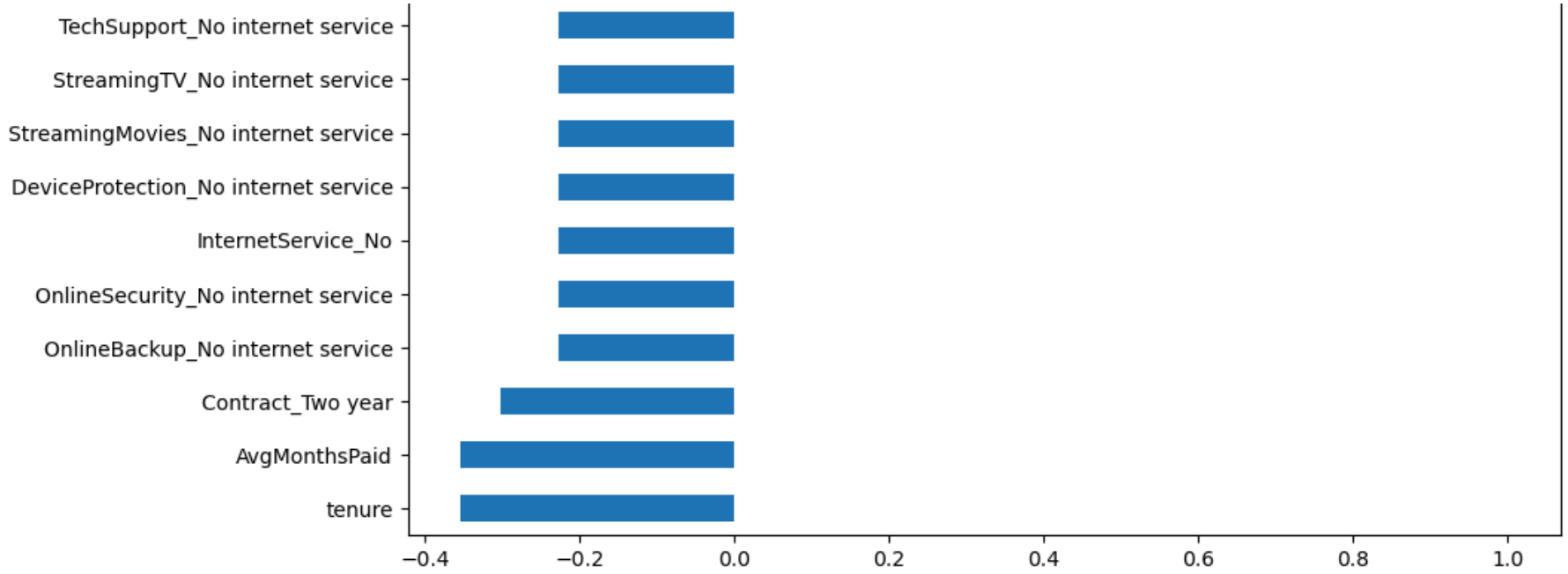
In [40]:
```
corr = df_final.corr()['Churn']

corr = corr.drop('Churn')

# Sorting
corr = corr.sort_values()

plt.figure(figsize=(10, 15))
corr.plot(kind='barh')
plt.title('Feature Correlation with Churn')
plt.show()
```


Feature Correlation with Churn
Churn_Binary
InternetService_Fiber optic
PaymentMethod_Electronic check
MonthlyCharges
PaperlessBilling_Yes
SeniorCitizen
StreamingTV_Yes
StreamingMovies_Yes
MultipleLines_Yes
PhoneService_Yes
gender_Male
MultipleLines_No phone service
DeviceProtection_Yes
OnlineBackup_Yes
PaymentMethod_Mailed check
PaymentMethod_Credit card (automatic)
Partner_Yes
Dependents_Yes
TechSupport_Yes
OnlineSecurity_Yes
Contract_One year
TotalCharges



11. Scaling & Preprocessing & Train/Test Split

In [44]:
```
# Split X and y
X = df_final.drop('Churn', axis=1)
y = df_final['Churn']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

In [46]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)

# Transform test
X_test = scaler.transform(X_test)

# Checking
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (5625, 32)
X_test shape: (1407, 32)
y_train shape: (5625,)
y_test shape: (1407,)
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: